

claude-windows / 9f6357d8-0e35-491a-a830-052e094c7f37

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9f6357d8-0e35-491a-a830-052e094c7f37\subagents\workflows\wf_e3cc36d8-eca\agent-ac983bb0c0e0b50c3.jsonl`  
- SHA-256: `8554a0f31ba30b44fdd35cd553536a3104691c79cbece1272e5cfc3c3ee715dc`  
- Source modified: `2026-06-21T09:19:20+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_e3cc36d8-eca`  
- session_id: `9f6357d8-0e35-491a-a830-052e094c7f37`
```

Transcript

1. user (2026-06-21T09:17:04.621Z)

In the khelsutra SPA at C:/Users/avidu/Projects/khelsutra/web, map what a B-P2 "ingest + progress" screen needs. The back-half already works: the SPA reads ?video=<id> ? POST /api/query ? renders rallies ? POST /api/compile ? download. The gap: there is NO UI to enter a bucket URI, call POST /api/process, and poll GET /api/jobs/{id} until result.video_id, then load that video_id.

Read: web/src/App.tsx (the ?video= flow + state), web/src/main.tsx, web/src/lib/ (api client, query, format), web/src/components/ (QueryBar, CompileResult, the language switcher), web/src/i18n* and the locales catalogs (web/src/locales or khelsutra/locales en/kn/hi common.json), web/package.json.

Produce a concrete, code-grounded B-P2 sketch: where the ingest input + submit + poll + progress live, how to reuse the existing API-call pattern, how the new view hands off to the existing rally-loading path (set video_id / navigate ?video=), and the new en/kn/hi i18n keys needed (matching existing key conventions). Note risks (polling/backoff, error mapping, offline banner interplay). Return structured JSON.

2. assistant (2026-06-21T09:17:06.904Z)

I'll map the B-P2 ingest+progress screen needs. Let me start by pulling the khelsutra repo fresh and exploring the structure.

3. user (2026-06-21T09:17:10.856Z)

Already up to date.

```
---BRANCH---  
master  
---STATUS---  
?? .claude/  
?? site/access.db
```

4. user (2026-06-21T09:17:11.184Z)

```
total 195  
drwxr-xr-x 1 avidu 197609      0 Jun 21 12:31 .  
drwxr-xr-x 1 avidu 197609      0 Jun 21 12:12 ..  
-rw-r--r-- 1 avidu 197609    41 Jun 20 15:25 .gitignore  
-rw-r--r-- 1 avidu 197609  1427 Jun 20 15:25 README.md  
drwxr-xr-x 1 avidu 197609      0 Jun 21 12:52 dist  
-rw-r--r-- 1 avidu 197609   303 Jun 20 15:25 index.html  
drwxr-xr-x 1 avidu 197609      0 Jun 21 12:27 node_modules  
-rw-r--r-- 1 avidu 197609 127996 Jun 21 12:31 package-lock.json  
-rw-r--r-- 1 avidu 197609   1074 Jun 21 12:31 package.json  
drwxr-xr-x 1 avidu 197609      0 Jun 21 12:31 src
```

```

-rw-r--r-- 1 avidu 197609    599 Jun 21 12:25 tsconfig.json
-rw-r--r-- 1 avidu 197609   1070 Jun 21 12:12 vite.config.ts
---SRC---
total 41
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:31 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:31 ..
-rw-r--r-- 1 avidu 197609  12752 Jun 21 12:25 App.tsx
drwxr-xr-x 1 avidu 197609    0 Jun 20 15:25 api
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:43 components
drwxr-xr-x 1 avidu 197609    0 Jun 20 22:21 hooks
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:56 i18n
-rw-r--r-- 1 avidu 197609    806 Jun 21 12:31 index.css
drwxr-xr-x 1 avidu 197609    0 Jun 20 22:45 lib
-rw-r--r-- 1 avidu 197609    704 Jun 21 12:31 main.tsx
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:12 test
---LIB---
total 68
drwxr-xr-x 1 avidu 197609    0 Jun 20 22:45 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:31 ..
-rw-r--r-- 1 avidu 197609   2157 Jun 20 22:34 floor.test.ts
-rw-r--r-- 1 avidu 197609   1755 Jun 20 22:34 floor.ts
-rw-r--r-- 1 avidu 197609   1637 Jun 20 22:45 format.test.ts
-rw-r--r-- 1 avidu 197609   1057 Jun 20 22:45 format.ts
-rw-r--r-- 1 avidu 197609   1144 Jun 20 15:25 log.ts
-rw-r--r-- 1 avidu 197609  14765 Jun 20 22:21 query.test.ts
-rw-r--r-- 1 avidu 197609  14666 Jun 20 22:21 query.ts
-rw-r--r-- 1 avidu 197609   1493 Jun 20 20:12 selection.test.ts
-rw-r--r-- 1 avidu 197609   1401 Jun 20 20:12 selection.ts
---COMPONENTS---
total 36
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:43 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:31 ..
-rw-r--r-- 1 avidu 197609   2351 Jun 21 12:25 CompileResult.tsx
-rw-r--r-- 1 avidu 197609   1991 Jun 21 12:43 LanguageSwitcher.tsx
-rw-r--r-- 1 avidu 197609   1100 Jun 20 23:14 QueryBar.examples.test.ts
-rw-r--r-- 1 avidu 197609   4216 Jun 21 12:25 QueryBar.tsx
-rw-r--r-- 1 avidu 197609   3366 Jun 21 12:25 RallyGrid.tsx
-rw-r--r-- 1 avidu 197609   3381 Jun 21 12:25 SelectionFooter.tsx

```

5. assistant (2026-06-21T09:17:13.925Z)

Repo is fresh on master. Let me read the core files: App.tsx, main.tsx, the api client, and the components.

6. user (2026-06-21T09:17:14.684Z)

```

1    import { useEffect, useMemo, useState } from "react";
2    import { useTranslation, Trans } from "react-i18next";
3    import { useRallySelection } from "../hooks/useRallySelection";
4    import { RallyGrid } from "../components/RallyGrid";
5    import { QueryBar } from "../components/QueryBar";
6    import { SelectionFooter } from "../components/SelectionFooter";
7    import { LanguageSwitcher } from "../components/LanguageSwitcher";
8    import { CompileResult, type BuildState } from "../components/CompileResult";
9    import { runQuery, type ParsedQuery } from "../lib/query";
10   import { floorRisk } from "../lib/floor";
11   import { reelFilename } from "../lib/format";
12   import { log, newCorrelationId } from "../lib/log";
13   import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount }
from "../api/client";
14   import type { EngineConfig, RallySegment } from "../api/types";
15
16   /** One compact, structured description of how a query was understood (for the trace
log). */
17   function parseTrace(q: ParsedQuery) {

```

```

18     return {
19         raw: q.raw,
20         recognized: q.recognized,
21         filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
22         sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
23         limit: q.limit ?? null,
24         unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
25     };
26 }
27
28 // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
29 // contract; only honest fields are surfaced. A real session loads from POST /api/query.
30 const DEMO_RALLIES: RallySegment[] = [
31     { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
32     { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
33     { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
34     { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: {} },
35     { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
36     { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
37 ];
38
39 type LoadState =
40     | { status: "demo" }
41     | { status: "loading" }
42     | { status: "loaded"; count: number }
43     | { status: "error"; error: string };
44
45 export default function App() {
46     const { t } = useTranslation();
47     // Job-tethered (D10): a finished job's video_id arrives via ?video=<id>; absent ?
demo data.
48     const videoId = useMemo(() => new
URLSearchParams(window.location.search).get("video"), []);
49
50     const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
51     const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
52     const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
53     const [reelName, setReelName] = useState("highlights");
54     const [build, setBuild] = useState<BuildState>({ status: "idle" });
55
56     // Display order is separate from selection: a query can re-sort the cards while the
shared
57     // selection set (sel) stays the single source of truth for what lands in the reel.
58     const sel = useRallySelection(rallies, "all");
59
60     // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
61     useEffect(() => {
62         if (!videoId) return;
63         const cid = newCorrelationId();
64         log("info", "rallies.load_start", { cid, video_id: videoId });
65         let alive = true;
66         listRallies(videoId, cid)
67             .then((rs) => {
68                 if (!alive) return;
69                 setRallies(rs);
70                 setOrder(rs);
71                 sel.apply(rs.map((r) => r.id));

```

```

72         setLoadState({ status: "loaded", count: rs.length });
73         log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
74     })
75     .catch((err) => {
76         if (!alive) return;
77         setLoadState({ status: "error", error: String(err) });
78         log("error", "rallies.load_failed", { cid, video_id: videoId, error: String(err)
79     });
80     return () => {
81         alive = false;
82     };
83     // videoId is stable for the page; sel.apply is a stable callback.
84     // eslint-disable-next-line react-hooks/exhaustive-deps
85     }, [videoId]);
86
87     // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is
88     // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
89     const [cfg, setCfg] = useState<EngineConfig | null>(null);
90     useEffect(() => {
91         let alive = true;
92         getConfig()
93             .then((c) => alive && setCfg(c))
94             .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
95         return () => {
96             alive = false;
97         };
98     }, []);
99     const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
100
101     // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
102     const ordinalOf = useMemo(() => {
103         const m = new Map<number, number>();
104         [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
105         return (id: number) => m.get(id) ?? 0;
106     }, [rallies]);
107
108     // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
109     const onBuild = async () => {
110         if (build.status === "building") return;
111         const cid = newCorrelationId();
112         const filename = reelFilename(reelName);
113         const ids = [...sel.selected];
114         if (!videoId) {
115             log("info", "build.no_video", { cid, selected: ids.length });
116             setBuild({
117                 status: "error",
118                 error: t("build.noVideo"),
119             });
120             return;
121         }
122         setBuild({ status: "building" });
123         log("info", "build.start", {
124             cid,
125             video_id: videoId,
126             filename,
127             selected: ids.length,
128             total_sec: Math.round(sel.totalSec),
129             below_floor: warning?.atRisk.length ?? 0,
130         });
131         try {

```

```

132         const res = await compileReel(videoId, ids, filename, cid);
133         const url = highlightUrl(videoId, filename);
134         setBuild({ status: "done", url, filename });
135         log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
136     } catch (err) {
137         const status = err instanceof ApiError ? err.status : undefined;
138         const detail = err instanceof ApiError && err.body ? parseDetail(err.body) :
String(err);
139         setBuild({ status: "error", error: detail });
140         log("error", "build.failed", { cid, video_id: videoId, filename, status, error:
String(err) });
141     }
142 };
143
144 // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
145 // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
146 // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
147 const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
148     const cid = newCorrelationId();
149     log("info", "query.parse", { cid, source, ...parseTrace(q) });
150
151     // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
152     // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
153     if (q.unavailable.length > 0) {
154         log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
155     }
156
157     if (!q.recognized) {
158         log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
159         return; // empty / unparseable ? leave the user's curation untouched
160     }
161
162     try {
163         const { ordered, selectedIds } = runQuery(rallies, q);
164         setOrder(ordered);
165         sel.apply(selectedIds);
166         log("info", "query.apply", {
167             cid,
168             source,
169             filters: q.filters.length,
170             sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
171             limit: q.limit ?? null,
172             total: rallies.length,
173             selected: selectedIds.length,
174             selected_ids: selectedIds,
175             dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
176         });
177         if (selectedIds.length === 0) {
178             // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
179             log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
180         }
181     } catch (err) {
182         // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
183         log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
184     }

```

```

185     };
186
187     // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
188     story.
189     const onToggle = (id: number) => {
190         log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
191         sel.toggle(id);
192     };
193     const onBulk = (op: "select_all" | "clear" | "invert") => {
194         const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
195         rallies.length - sel.count;
196         log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
197         resultCount });
198         if (op === "select_all") sel.selectAll();
199         else if (op === "clear") sel.clearAll();
200         else sel.invert();
201     };
202     return (
203         <main className="min-h-screen bg-paper text-ink pb-24">
204             <header className="bg-ink text-white">
205                 <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
206                 py-4">
207                     <span className="text-xl font-extrabold tracking-tight">
208                         Khel<span className="text-green">?</span>Sutra <span className="font-
209                         semibold text-lime">Studio</span>
210                     </span>
211                     <LanguageSwitcher />
212                 </div>
213             </header>
214
215             <section className="mx-auto max-w-5xl px-6 py-10">
216                 <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
217                 semibold text-green">
218                     {t("app.badge")}
219                 </span>
220                 <h1 className="mt-4 text-3xl font-extrabold tracking-
221                 tight">{t("app.title")}</h1>
222                 <p className="mt-2 max-w-2xl text-ink/70">
223                     <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
224                 </p>
225
226                 <ModeBanner load={loadState} videoId={videoId} />
227
228                 <div className="mt-6">
229                     <QueryBar onApply={onApplyQuery} />
230                 </div>
231
232                 <div className="mt-6 flex flex-wrap gap-2 text-sm">
233                     <button type="button" onClick={() => onBulk("select_all")} className="rounded-
234                     full border border-black/15 px-3 py-1 hover:border-green">{t("bulk.selectAll")}</button>
235                     <button type="button" onClick={() => onBulk("clear")} className="rounded-full
236                     border border-black/15 px-3 py-1 hover:border-green">{t("bulk.clear")}</button>
237                     <button type="button" onClick={() => onBulk("invert")} className="rounded-full
238                     border border-black/15 px-3 py-1 hover:border-green">{t("bulk.invert")}</button>
239                 </div>
240
241                 <div className="mt-5">
242                     <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
243                     ordinalOf={ordinalOf} />
244                 </div>
245
246                 <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })} />
247             </section>
248         </main>

```

```

239     <SelectionFooter
240         count={sel.count}
241         totalSec={sel.totalSec}
242         reelName={reelName}
243         onReelNameChange={setReelName}
244         onBuild={onBuild}
245         building={build.status === "building"}
246         warning={warning}
247     />
248 </main>
249 );
250 }
251
252 /** Pull FastAPI's {"detail": "..."} message out of an error body, else show it raw. */
253 function parseDetail(body: string): string {
254     try {
255         const j = JSON.parse(body);
256         if (j && typeof j.detail === "string") return j.detail;
257     } catch {
258         /* not JSON ? fall through */
259     }
260     return body;
261 }
262
263 function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
264     if (load.status === "demo") {
265         return (
266             <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
267                 <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
268             </p>
269         );
270     }
271     if (load.status === "loading") {
272         return (
273             <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
274                 <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code />
275             </p>
276         );
277     }
278     if (load.status === "error") {
279         return (
280             <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
281 text-red-800" data-testid="mode-banner">
282                 <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
283 components={{ code: <code /> }} />
284             </p>
285         );
286     }
287     return (
288         <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
289             <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
290 <strong className="text-ink/70" /> }} />
291             </p>
292         );
293     }
294 }

```

7. user (2026-06-21T09:17:15.143Z)

```

1     import { StrictMode } from "react";
2     import { createRoot } from "react-dom/client";
3     import App from "../App";
4     import "../i18n"; // initialize the i18next singleton before the app renders (tracker B1)
5     // Self-hosted OFL Noto fonts so Kannada/Hindi render crisply offline (tracker B4 /

```

```

decision D7) ?
6      // no Google Fonts runtime dependency. unicode-range-scoped, so only Indic codepoints
use them.
7      import "@fontsource-variable/noto-sans-kannada/index.css";
8      import "@fontsource-variable/noto-sans-devanagari/index.css";
9      import "./index.css";
10
11     const root = document.getElementById("root");
12     if (!root) throw new Error("#root not found");
13
14     createRoot(root).render(
15       <StrictMode>
16         <App />
17       </StrictMode>,
18     );
19

```

8. user (2026-06-21T09:17:16.055Z)

```

api/:
total 16
drwxr-xr-x 1 avidu 197609    0 Jun 20 15:25 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:31 ..
-rw-r--r-- 1 avidu 197609 3351 Jun 20 15:25 client.test.ts
-rw-r--r-- 1 avidu 197609 3754 Jun 20 15:25 client.ts
-rw-r--r-- 1 avidu 197609 1526 Jun 20 15:25 types.ts

hooks/:
total 8
drwxr-xr-x 1 avidu 197609    0 Jun 20 22:21 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:31 ..
-rw-r--r-- 1 avidu 197609 1334 Jun 20 22:21 useRallySelection.ts

i18n/:
total 28
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:56 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:31 ..
-rw-r--r-- 1 avidu 197609 4433 Jun 21 12:56 hygiene.test.ts
-rw-r--r-- 1 avidu 197609 7027 Jun 21 12:43 i18n.test.tsx
-rw-r--r-- 1 avidu 197609 2851 Jun 21 12:43 index.ts
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:43 locales

```

9. assistant (2026-06-21T09:17:18.090Z)

Now let me read the API client, types, and the i18n setup with locales.

10. user (2026-06-21T09:17:19.056Z)

```

1      // Typed client for the engine FastAPI. SAME-ORIGIN relative `/api/*` (the appliance
serves
2      // UI + engine behind one origin; in dev Vite proxies `/api` ? :8000). Every call
carries a
3      // correlation id (header + logs) so a journey is traceable end to end.
4      import { log, newCorrelationId } from "../lib/log";
5      import type { RallySegment, QueryResponse, CompileResponse, EngineConfig } from
"./types";
6
7      const BASE = "/api";
8
9      export class ApiError extends Error {
10         readonly cid?: string;
11         readonly status?: number;
12         readonly body?: string;
13         constructor(message: string, opts: { cid?: string; status?: number; body?: string;

```

```

cause?: unknown } = {}) {
14     super(message);
15     this.name = "ApiError";
16     this.cid = opts.cid;
17     this.status = opts.status;
18     this.body = opts.body;
19     if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
20 }
21 }
22
23 interface RequestOpts {
24     method?: string;
25     body?: string;
26     headers?: Record<string, string>;
27     /** Correlation id to thread through this call; auto-generated if absent. */
28     cid?: string;
29 }
30
31 async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
32     const cid = opts.cid ?? newCorrelationId();
33     const url = `${BASE}${path}`;
34     const method = opts.method ?? "GET";
35     log("info", "api.request", { cid, method, url });
36
37     let res: Response;
38     try {
39         res = await fetch(url, {
40             method,
41             body: opts.body,
42             headers: { "content-type": "application/json", "x-correlation-id": cid,
... (opts.headers ?? {}) },
43         });
44     } catch (err) {
45         // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
46         log("error", "api.network_error", { cid, method, url, error: String(err) });
47         throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
48     }
49
50     if (!res.ok) {
51         const body = await res.text().catch(() => "");
52         log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
53         throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
54     }
55
56     const data = (await res.json()) as T;
57     log("info", "api.response", { cid, method, url, status: res.status });
58     return data;
59 }
60
61 /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
62 export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
63     const data = await request<QueryResponse>("/query", {
64         method: "POST",
65         body: JSON.stringify({ video_id: videoId }),
66         cid,
67     });
68     return data.play_segments ?? [];
69 }
70
71 /** Stitch selected rallies into one reel. POST /api/compile {video_id, segment_ids,
output_filename} [main.py:342]. */

```

```

72     export async function compileReel(
73         videoId: string,
74         segmentIds: number[],
75         outputFilename: string,
76         cid?: string,
77     ): Promise<CompileResponse> {
78         return request<CompileResponse>("/compile", {
79             method: "POST",
80             body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename }),
81             cid,
82         });
83     }
84
85     /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
86     export function highlightUrl(videoId: string, filename: string): string {
87         return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
88     }
89
90     /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
91     export async function getConfig(cid?: string): Promise<EngineConfig> {
92         return request<EngineConfig>("/config", { method: "GET", cid });
93     }
94
95     /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
96     export function minShotCount(cfg: EngineConfig): number | undefined {
97         return cfg.stitching?.min_shot_count;
98     }
99

```

11. user (2026-06-21T09:17:19.524Z)

```

1     // Types mirror the engine's verified contract (badminton-highlight-indexer).
2     // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3     // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5     export interface RallyMetadata {
6         /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7         shot_count?: number;
8         /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9         shot_count_confidence?: string;
10        detector?: string;
11        [key: string]: unknown;
12    }
13
14    export interface RallySegment {
15        /** The integer the UI selects and passes to /api/compile as segment_ids. */
16        id: number;
17        start_time: number;
18        end_time: number;
19        duration: number;
20        /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21        server: string | null;
22        /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23        receiver: string | null;
24        metadata: RallyMetadata;
25        /** motion-only fabricated sub-events ? DO NOT surface. */
26        events?: unknown[];
27    }
28

```

```

29     export interface QueryResponse {
30         play_segments: RallySegment[];
31     }
32
33     export interface CompileResponse {
34         status: string;
35         /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
36         filepath: string;
37     }
38
39     export interface EngineConfig {
40         stitching?: {
41             /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
42             */
43             min_shot_count?: number;
44             [key: string]: unknown;
45         };
46         [key: string]: unknown;
47     }

```

12. user (2026-06-21T09:17:20.020Z)

```

1      // i18n scaffold (tracker B1) ? the shared localization foundation for KhelSutra Studio.
2      //
3      // Per docs/FRIENDLY_AND_MULTILINGUAL.md + the engine's I18N_PLAN.md:
4      //   - keys, never prose (t("app.title"), enforced later by a no-bare-strings lint, B2);
5      //   - ICU MessageFormat for interpolation/plurals (Kannada/Hindi plural rules live in
6      the catalog,
7      //       not the code ? replaces the SPA's hand-rolled `count > 1 ? "s" : ""`);
8      //   - one negotiation order shared with the marketing site (B3);
9      //   - always-fallback to `en` so a missing translation degrades to English, never a raw
10     key.
11     import i18n from "i18next";
12     import { initReactI18next } from "react-i18next";
13     import LanguageDetector from "i18next-browser-languagedetector";
14     import ICU from "i18next-icu";
15     import enCommon from "../locales/en/common.json";
16     import knCommon from "../locales/kn/common.json";
17     import hiCommon from "../locales/hi/common.json";
18
19     // Supported locales (D6): `en` + `kn` + `hi`. Anything else falls back to `en`.
20     export const SUPPORTED_LOCALES = ["en", "kn", "hi"] as const;
21     export type Locale = (typeof SUPPORTED_LOCALES)[number];
22     export const DEFAULT_LOCALE: Locale = "en";
23
24     // ONE persisted-preference key, shared with the marketing site's t() shim (D9 / tracker
25     B3), so a
26     // visitor's language choice carries across both surfaces.
27     export const LANG_STORAGE_KEY = "khelsutra.lang";
28
29     // Catalogs are co-located here (D9). `kn` + `hi` shipped (B5/B6, AI-drafted ? native
30     review
31     // pending). A missing key in any locale still falls back to `en` ? the by-design
32     invariant.
33     const resources = {
34         en: { common: enCommon },
35         kn: { common: knCommon },
36         hi: { common: hiCommon },
37     } as const;
38
39     void i18n
40         .use(ICU)
41         .use(LanguageDetector)
42         .use(initReactI18next)

```

```

38     .init({
39         resources,
40         fallbackLng: DEFAULT_LOCALE,
41         supportedLngs: [...SUPPORTED_LOCALES],
42         nonExplicitSupportedLngs: true, // en-IN -> en
43         defaultNS: "common",
44         ns: ["common"],
45         interpolation: { escapeValue: false }, // React already escapes
46         returnNull: false,
47         // Negotiation order (I18N_PLAN §2.3): persisted pref -> ?lang= -> browser -> en.
48         detection: {
49             order: ["localStorage", "querystring", "navigator"],
50             lookupLocalStorage: LANG_STORAGE_KEY,
51             lookupQuerystring: "lang",
52             caches: ["localStorage"],
53         },
54         react: { useSuspense: false },
55     });
56
57     // Keep <html lang> in sync with the active locale (screen readers + SEO use it).
Reflects the
58     // resolved content language, so an `en` fallback for an absent locale still reads as
`en`.
59     function syncDocumentLang(lng: string) {
60         if (typeof document !== "undefined") {
61             document.documentElement.lang = (i18n.resolvedLanguage || lng ||
DEFAULT_LOCALE).split("-")[0];
62         }
63     }
64     i18n.on("languageChanged", syncDocumentLang);
65     syncDocumentLang(i18n.language ?? DEFAULT_LOCALE);
66
67     export default i18n;
68

```

13. user (2026-06-21T09:17:21.282Z)

```

total 4
drwxr-xr-x 1 avidu 197609 0 Jun 21 12:43 .
drwxr-xr-x 1 avidu 197609 0 Jun 21 12:56 ..
drwxr-xr-x 1 avidu 197609 0 Jun 21 13:00 en
drwxr-xr-x 1 avidu 197609 0 Jun 21 13:00 hi
drwxr-xr-x 1 avidu 197609 0 Jun 21 13:00 kn
===
./en/common.json
./hi/common.json
./kn/common.json

```

14. assistant (2026-06-21T09:17:23.585Z)

Now let me read the locale catalogs, the QueryBar, CompileResult, LanguageSwitcher components, and package.json. Also need to check the engine's /api/process and /api/jobs contract.

15. user (2026-06-21T09:17:25.019Z)

```

1     {
2         "app": {
3             "badge": "per-rally selection · alpha",
4             "title": "Pick the rallies for your reel",
5             "intro": "Filter by <b>rally length</b> ? e.g. <i>?top 15 longest?</i> ? then fine-
tune by tapping cards. Everything starts <b>selected</b>; just drop the ones you don't want."
6         },
7         "lang": {
8             "label": "Language",

```

```

9      "en": "English",
10     "kn": "?????",
11     "hi": "??????"
12 },
13 "bulk": {
14     "selectAll": "Select all",
15     "clear": "Clear",
16     "invert": "Invert"
17 },
18 "mode": {
19     "demo": "Demo data ? open with <code>?video=?video_id?</code> against a running
engine to load a real session.",
20     "loading": "Loading rallies for <code>{videoId}</code>?",
21     "error": "Couldn't load rallies for <code>{videoId}</code>: {error}",
22     "loaded": "Loaded <b>{count}</b> rallies from this session."
23 },
24 "query": {
25     "placeholder": "e.g. over 10s · top 15 longest · shortest first",
26     "arialLabel": "Filter rallies by length, count or order",
27     "apply": "Apply",
28     "notAvailable": "not available yet",
29     "notAvailableTitle": "?{token}? needs {kind} detection ? not available yet",
30     "unreadable": "Couldn't read that ? try ?over 10s? or ?top 15 longest?.",
31     "hint": "Type over 10s for rallies longer than 10 seconds ? or tap a sample below.",
32     "try": "Sample queries:",
33     "support": "Filtering by <b>rally length</b> is fully supported. Shot-count is
<x>experimental</x> (under development).",
34 },
35 "grid": {
36     "empty": "No rallies to show yet.",
37     "rally": "Rally {n}",
38     "inReel": "Rally {n}, in reel",
39     "notInReel": "Rally {n}, not in reel",
40     "start": "start",
41     "long": "long",
42     "shots": "~{count} shots · experimental",
43     "shotsTitle": "Shot count is experimental (under development) and may be
inaccurate{conf}. Filter by rally length instead.",
44     "shotsConfidence": " ? confidence: {conf}"
45 },
46 "footer": {
47     "count": "{count, plural, one {# rally} other {# rallies}}",
48     "summaryRest": "selected · {total} total",
49     "reelName": "Reel name",
50     "reelNamePlaceholder": "my-highlights",
51     "build": "Build reel ({count})",
52     "building": "Building?",
53     "floorOne": "<b>Your only selected rally</b> has fewer than {min} shots ? the engine
would skip it, so the build would fail. Pick a longer rally.",
54     "floorAll": "<b>All {count} selected rallies</b> have fewer than {min} shots ? the
engine skips every one, so the build would fail. Pick longer rallies.",
55     "floorSomeLead": "{count, plural, one {# rally} other {# rallies}}",
56     "floorSomeRest": "with fewer than {min} shots will be skipped when building (the
engine's <code>min_shot_count</code> floor).",
57 },
58 "result": {
59     "ready": "Your reel is ready",
60     "failed": "Build failed",
61     "dismiss": "Dismiss",
62     "download": "Download {filename}"
63 },
64 "build": {
65     "noVideo": "Open with ?video=<video_id> against a running engine to build a reel ?
demo data can't be compiled."
66 },

```



```

50         className="min-w-[16rem] flex-1 rounded-xl border border-black/15 bg-white
px-4 py-2 text-sm text-ink outline-none focus:border-green"
51     />
52     <button
53         type="submit"
54         disabled={!parsed.recognized}
55         className="rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-white
disabled:opacity-40"
56     >
57         {t("query.apply")}
58     </button>
59 </form>
60
61     {/* Live interpretation ? transparent + hand-correctable (no hidden NL guessing).
*/}
62     <div aria-live="polite" data-testid="query-pill" className="mt-2 flex flex-wrap
items-center gap-1.5">
63         {clauses.map((c) => (
64             <span key={c} className={CHIP + " bg-green/15 text-green"}>
65                 {c}
66             </span>
67         ))}
68         {parsed.unavailable.map((u) => (
69             <span
70                 key={u.token}
71                 title={t("query.notAvailableTitle", { token: u.token, kind: u.kind })}
72                 className={CHIP + " bg-black/5 text-ink/45"}
73             >
74                 <span className="line-through">{u.token}</span> · {t("query.notAvailable")}
75             </span>
76         ))}
77         {typed && !parsed.recognized && parsed.unavailable.length === 0 && (
78             <span className="text-xs text-ink/50">{t("query.unreadable")}</span>
79         )}
80         {!typed && <span className="text-xs text-ink/50">{t("query.hint")}</span>}
81     </div>
82
83     <div className="mt-3 flex flex-wrap gap-2 text-sm">
84         <span className="text-ink/40">{t("query.try")}</span>
85         {EXAMPLES.map((ex) => (
86             <button
87                 key={ex}
88                 type="button"
89                 onClick={() => applyText(ex)}
90                 className="rounded-full border border-black/15 px-3 py-1 text-xs
hover:border-green"
91             >
92                 {ex}
93             </button>
94         ))}
95     </div>
96
97     <p className="mt-2 text-xs text-ink/40">
98         <Trans i18nKey="query.support" components={{ b: <strong className="text-ink/60"
/>, x: <span className="text-amber-700" /> }} />
99     </p>
100 </div>
101 );
102 }
103

```

17. user (2026-06-21T09:17:25.957Z)

```

1      // The compiled-reel result (PER_RALLY_SELECTION.md P5): once Build ? POST /api/compile
succeeds,

```

```

2      // preview the reel inline (<video> over GET /api/highlights/...) and offer a download.
The engine
3      // returns a server-local filepath, NOT a URL (main.py:394), so the SPA builds the URL
itself. On a
4      // failure (e.g. the min_shot_count 400) we show the engine's reason ? loud, never
swallowed.
5
6      import { useTranslation } from "react-i18next";
7
8      export type BuildState =
9          | { status: "idle" }
10         | { status: "building" }
11         | { status: "done"; url: string; filename: string }
12         | { status: "error"; error: string };
13
14     interface Props {
15         build: BuildState;
16         onDismiss: () => void;
17     }
18
19     export function CompileResult({ build, onDismiss }: Props) {
20         const { t } = useTranslation();
21         if (build.status === "idle" || build.status === "building") return null;
22
23         return (
24             <section
25                 data-testid="compile-result"
26                 className={
27                     "mt-6 rounded-xl border p-4 " +
28                     (build.status === "error" ? "border-red-200 bg-red-50" : "border-green/30 bg-
green/5")
29                 }
30             >
31                 <div className="flex items-start justify-between gap-3">
32                     <h2 className="font-semibold text-ink">
33                         {build.status === "done" ? t("result.ready") : t("result.failed")}
34                     </h2>
35                     <button
36                         type="button"
37                         onClick={onDismiss}
38                         aria-label={t("result.dismiss")}
39                         className="rounded-full px-2 text-ink/50 hover:text-ink"
40                     >
41                         ?
42                     </button>
43                 </div>
44
45                 {build.status === "done" ? (
46                     <div className="mt-3">
47                         {/* eslint-disable-next-line jsx-a11y/media-has-caption ? a stitched rally
reel has no captions */}
48                         <video
49                             data-testid="reel-video"
50                             src={build.url}
51                             controls
52                             playsInline
53                             className="w-full max-w-xl rounded-lg bg-black"
54                         />
55                         <a
56                             href={build.url}
57                             download={build.filename}
58                             className="mt-3 inline-block rounded-full bg-green px-4 py-2 text-sm font-
semibold text-[#04140D]"
59                         >
60                             {t("result.download", { filename: build.filename })}

```

```

61         </a>
62     </div>
63     ) : (
64         <p className="mt-2 text-sm text-red-800" data-testid="compile-error">
65             {build.error}
66         </p>
67     )}
68 </section>
69 );
70 }
71

```

18. user (2026-06-21T09:17:26.431Z)

```

1     import { useTranslation } from "react-i18next";
2     import { SUPPORTED_LOCALES, DEFAULT_LOCALE, type Locale } from "../i18n";
3
4     // Each locale's label is its own autonym, shown in its own script. Kannada renders
crisply once the
5     // bundled Noto Sans Kannada font lands (tracker B4); most devices already ship a system
Kannada font.
6     const LANG_LABEL_KEY: Record<Locale, string> = {
7         en: "lang.en",
8         kn: "lang.kn",
9         hi: "lang.hi",
10    };
11
12    /**
13     * A persistent language switcher. Selecting a locale calls i18next's changeLanguage,
which the
14     * detector caches to localStorage (LANG_STORAGE_KEY) ? so the choice is the top-
priority signal on
15     * the next visit (negotiation: persisted pref wins). Icon-free + non-color active state
via
16     * aria-pressed keeps it usable for non-tech-savvy users (FRIENDLY_AND_MULTILINGUAL §4).
17     */
18    export function LanguageSwitcher() {
19        const { t, i18n } = useTranslation();
20        // Highlight the user's CHOSEN locale (i18n.language), not the resolved-for-content
one:
21        // resolvedLanguage falls back to `en` when a catalog is missing, so picking Kannada
(content
22        // still English until the kn catalog lands, B5) must still show Kannada as selected.
Compare on
23        // the base subtag so a detected "en-IN" highlights "en".
24        const active = (i18n.language || DEFAULT_LOCALE).split("-")[0];
25
26        return (
27            <nav aria-label={t("lang.label")} className="flex items-center gap-1">
28                {SUPPORTED_LOCALES.map((loc) => {
29                    const isActive = active === loc;
30                    return (
31                        <button
32                            key={loc}
33                            type="button"
34                            lang={loc}
35                            aria-pressed={isActive}
36                            onClick={() => void i18n.changeLanguage(loc)}
37                            className={[
38                                "rounded-full px-3 py-1.5 text-sm font-semibold transition",
39                                isActive ? "bg-white text-ink" : "text-white/70 hover:text-white",
40                            ].join(" ")}
41                        >
42                            {t(LANG_LABEL_KEY[loc])}
43                        </button>

```

```

44         );
45     }}}
46     </nav>
47 );
48 }
49

```

19. user (2026-06-21T09:17:26.862Z)

```

1  {
2      "name": "khelsutra-web",
3      "private": true,
4      "version": "0.0.0",
5      "type": "module",
6      "description": "KhelSutra Studio ? operator console SPA (drives the rally indexer
engine).",
7      "scripts": {
8          "dev": "vite",
9          "build": "vite build",
10         "preview": "vite preview",
11         "typecheck": "tsc --noEmit",
12         "test": "vitest run"
13     },
14     "dependencies": {
15         "@fontsource-variable/noto-sans-devanagari": "^5.2.8",
16         "@fontsource-variable/noto-sans-kannada": "^5.2.9",
17         "i18next": "^26.3.1",
18         "i18next-browser-languagedetector": "^8.2.1",
19         "i18next-icu": "^2.4.3",
20         "react": "^19.0.0",
21         "react-dom": "^19.0.0",
22         "react-i18next": "^17.0.8"
23     },
24     "devDependencies": {
25         "@tailwindcss/vite": "^4.0.0",
26         "@testing-library/dom": "^10.4.1",
27         "@testing-library/react": "^16.3.2",
28         "@types/node": "^26.0.0",
29         "@types/react": "^19.0.0",
30         "@types/react-dom": "^19.0.0",
31         "@vitejs/plugin-react": "^4.3.4",
32         "jsdom": "^29.1.1",
33         "tailwindcss": "^4.0.0",
34         "typescript": "^5.7.0",
35         "vite": "^6.0.0",
36         "vitest": "^3.0.0"
37     }
38 }
39

```

20. assistant (2026-06-21T09:17:31.096Z)

Now I have the full SPA picture. Let me check the engine's actual `/api/process` and `/api/jobs` contract in the indexer repo, plus the kn/hi catalogs to match key conventions, and the log/format helpers.

21. user (2026-06-21T09:17:36.317Z)

Your configuration specifies to merge with the ref 'refs/heads/feat/serving-m4-device-context' from the remote, but no such ref was fetched.

===PROCESS/JOB in main.py===

```

30:from backend.utils.hashing import compute_video_id # canonical file-identity hashing
91:class ProcessRequest(BaseModel):
98:    video_id: str

```

```

105:     video_id: str
109: def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
117:     db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
119:     db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
140: def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
143:     This is the former synchronous /api/process body, unchanged in behavior, now run on
160:     video_id = compute_video_id(local_video)
161:     was_reingest = db_instance.get_video(video_id) is not None
179:     db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump() for r
in val_results])
181:     "video_id": video_id,
190:     db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for r
in val_results])
198:     "video_id": video_id,
211:     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
214:     result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
219:     db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
225:     db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
227:     "video_id": video_id,
237:     _finalize_reingest(video_id, segments, play_wm, cand_wm)
240:     "video_id": video_id,
251:     db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
260: @app.post("/api/process", status_code=202)
261: def process_video(req: ProcessRequest):
262:     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
299:     content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
303: @app.get("/api/jobs/{job_id}")
304: def get_job(job_id: str):
315:     "video_id": job.get("video_id"),
336: @app.get("/api/highlights/{video_id}/{filename}")
337: def download_highlight(video_id: str, filename: str):
345:     if not db_instance.get_video(video_id):
368:     segments = db_instance.query_play_segments(req.video_id, filters)
373:     video = db_instance.get_video(req.video_id)
378:     all_segments = db_instance.query_play_segments(req.video_id, {})
425:     download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
609:     video_id = compute_video_id(local_video)
610:     was_reingest = db_instance.get_video(video_id) is not None
619:     db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r in
val_results])
645:     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
647:     result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
658:     _finalize_reingest(video_id, segments, play_wm, cand_wm)
662:     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,

```

22. user (2026-06-21T09:17:36.324Z)

```

1      // Structured client logger ? the observability seed for the operator console
2      // (docs/LOCAL_APPLIANCE_ARCHITECTURE.md §5). Every API call carries a CORRELATION ID so
a
3      // single user journey is traceable across the UI ? FastAPI ? worker. Logs are JSON-
shaped
4      // so they aggregate cleanly off a transient host. Keep this dependency-free.
5
6      export type LogLevel = "debug" | "info" | "warn" | "error";
7
8      /** A short, unique id to correlate all log lines (and the x-correlation-id header) for
one action. */
9      export function newCorrelationId(): string {
10         const c = (globalThis as { crypto?: Crypto }).crypto;
11         if (c && typeof c.randomUUID === "function") return c.randomUUID();

```

```

12         // Fallback for environments without crypto.randomUUID.
13         return `cid_${Date.now().toString(36)}_${Math.floor(Math.random() *
1e9).toString(36)}`;
14     }
15
16     export function log(level: LogLevel, event: string, fields: Record<string, unknown> =
{}): void {
17         const line = JSON.stringify({ ts: new Date().toISOString(), level, event, ...fields
});
18         if (level === "error") console.error(line);
19         else if (level === "warn") console.warn(line);
20         else console.log(line);
21     }
22

```

23. user (2026-06-21T09:17:36.327Z)

```

1         // Small pure formatters (node-testable, no React).
2
3         /** Seconds ? "m:ss" (e.g. 65 ? "1:05"). Rounds to the nearest second; clamps negatives
to 0. */
4         export function mmss(totalSeconds: number): string {
5             const s = Math.max(0, Math.round(totalSeconds || 0));
6             const m = Math.floor(s / 60);
7             return `${m}:${String(s % 60).padStart(2, "0")}`;
8         }
9
10        /** "3 rallies" / "1 rally" ? count with a correctly pluralized noun. */
11        export function pluralize(n: number, one: string, many = one + "s"): string {
12            return `${n} ${n === 1 ? one : many}`;
13        }
14
15        /** Turn a user reel name into a safe bare `.mp4` filename for POST /api/compile. The
engine's
16        * `safe_output_filename` (backend/utils/paths.py) rejects path separators / traversal /
absolute
17        * names, so we slugify to [a-z0-9-] and always end in `.mp4`. Empty ? "highlights.mp4".
*/
18        export function reelFilename(name: string): string {
19            const slug = (name || "")
20                .toLowerCase()
21                .replace(/^[a-z0-9-]+/g, "-")
22                .replace(/^-+|-+$/g, "")
23                .slice(0, 80);
24            return `${slug || "highlights"}.mp4`;
25        }
26

```

24. assistant (2026-06-21T09:17:42.963Z)

The indexer repo is on a feature branch with a pull config issue, but I can read the local main.py directly. Let me read the exact /api/process and /api/jobs contract, plus the ProcessRequest model and the kn/hi catalogs.

25. user (2026-06-21T09:17:44.170Z)

```

88         return cfg.model_dump(mode="json")
89
90     # --- API Models ---
91     class ProcessRequest(BaseModel):
92         video_path: str
93         player_pool: Optional[List[str]] = None
94         max_frames: Optional[int] = None
95         segmenter: Optional[str] = None

```

```

96
97     class QueryRequest(BaseModel):
98         video_id: str
99         server: Optional[str] = None
100         receiver: Optional[str] = None
101         duration_min: Optional[float] = None
102         event_type: Optional[str] = None
103
104     class CompileRequest(BaseModel):
105         video_id: str
106         segment_ids: List[int]
107         output_filename: str
108
109     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
110         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
111
112         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
113         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
114         and
115         keep the prior good results intact ? a failed/empty re-run never destroys prior
116         data.
117         """
118         if segments:
119             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)

```

26. user (2026-06-21T09:17:44.873Z)

```

260     @app.post("/api/process", status_code=202)
261     def process_video(req: ProcessRequest):
262         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
263
264         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
265         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
266         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
267         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
268         """
269         allowed_root = getattr(getattr(global_config, "security", None),
270 "allowed_video_root", None)
271         try:
272             validate_input_path(req.video_path, allowed_root=allowed_root)
273             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
274             unsupported
275             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
276             input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
277 "storage", None))
278             except ValueError as e:
279                 raise HTTPException(status_code=400, detail=str(e)) from e
280             # The local-existence fast-fail applies only to a LOCAL input, and stats the
281             RESOLVED local path
282             # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
283             string. A
284             # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
285             when the worker
286             # fetches it (a fetch miss fails the job loudly). validate_input_path above still
287             runs for every
288             # input: its null-byte guard always applies, and a configured allowed_video_root
289             (hosted mode)
290             # rejects a non-local URI as out-of-root.
291             if input_ref.backend == "local" and not os.path.exists(input_ref.key):
292                 raise HTTPException(status_code=404, detail=f"Video file not found at
293 '{req.video_path}'")
294             if req.segmenter and not segmenter_registry.get(req.segmenter):
295                 available = list(segmenter_registry.list_available().keys())
296                 raise HTTPException(
297                     status_code=400,

```

```

289         detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
290     )
291
292     job_id = uuid.uuid4().hex
293     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
294                                 req.player_pool, req.max_frames):
295         raise HTTPException(status_code=500, detail="Failed to persist job record.")
296     _get_executor().submit(_drain_due_jobs)
297     return JSONResponse(
298         status_code=202,
299         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
300     )
301
302
303 @app.get("/api/jobs/{job_id}")
304 def get_job(job_id: str):
305     """Job state: {status, progress, result, reports}. `status` = execution state
306     (queued|running|done|failed); the pipeline verdict is result["status"]."""
307     job = db_instance.get_job(job_id)
308     if not job:
309         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
310     result = job.get("result")
311     return {
312         "job_id": job["id"],
313         "status": job["status"],
314         "progress": job.get("progress"),
315         "video_id": job.get("video_id"),
316         "error": job.get("error"),
317         "result": result,
318         "reports": (result or {}).get("reports"),
319         "created_at": job.get("created_at"),
320         "started_at": job.get("started_at"),
321         "finished_at": job.get("finished_at"),
322     }
323
324 # --- Chunked upload (B2, PR-2) -----
325 # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
326 in
327 # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
328 via
329 # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
330 # sequential-part logic, and abort cleanup ? is unchanged.
331 from backend.api import uploads as uploads_api
332
333 app.include_router(uploads_api.router)
334
335 # --- Highlight download (B3, PR-2) -----

```

27. user (2026-06-21T09:17:45.306Z)

```

1  {
2      "app": {
3          "badge": "?????-???? ??? . ??????",
4          "title": "???? ??? ?? ??? ?????? ??????",
5          "intro": "<b>???? ?? ?????</b> ?? ??? ? ???? ? ???? <i>????? ???? ?
15?</i> ? ??? ????? ???? ???? ?????? ?? ????????? ???? ? ? ? <b>????</b> ???? ???? ?; ?? ?
???? ????? ?????? ??????"
6      },
7      "lang": {
8          "label": "????",
9          "en": "English",
10         "kn": "????",
11         "hi": "?????"

```

```

12     },
13     "bulk": {
14         "selectAll": "??? ??????",
15         "clear": "???? ??????",
16         "invert": "?????"
17     },
18     "mode": {
19         "demo": "????? ???? ? ???? ???? ??? ???? ?? ??? ???? ???? ?? ???
<code>?video=?video_id?</code> ?? ??????",
20         "loading": "<code>{videoId}</code> ?? ??? ??????? ???? ?? ??? ??????",
21         "error": "<code>{videoId}</code> ?? ??? ??????? ???? ???? ?? ?????: {error}",
22         "loaded": "? ? ???? ? <b>{count}</b> ?????? ???? ??????"
23     },
24     "query": {
25         "placeholder": "???. over 10s · top 15 longest · shortest first",
26         "ariaLabel": "?????, ?????? ?? ???? ?? ???? ?? ??????? ??????? ??????",
27         "apply": "???? ??????",
28         "notAvailable": "??? ?????? ??????",
29         "notAvailableTitle": "?{token}? ?? ??? {kind} ????? ?????? ? ??? ?????? ??????",
30         "unreadable": "? ? ???? ???? ? ?over 10s? ?? ?top 15 longest? ??????????",
31         "hint": "10 ?????? ?? ???? ?????????? ?? ??? over 10s ???? ???? ? ? ???? ?? ??? ??????
??? ??????",
32         "try": "????? ??????:",
33         "support": "<b>????? ?? ?????</b> ?? ???? ?? ??????? ???? ???? ?? ??????? ????
???-????? <x>????????????</x> ?? (?????????)?"
34     },
35     "grid": {
36         "empty": "?????? ?? ??? ??? ??? ???? ???? ????",
37         "rally": "???? {n}",
38         "inReel": "???? {n}, ??? ???",
39         "notInReel": "???? {n}, ??? ??? ????",
40         "start": "????",
41         "long": "????",
42         "shots": "~{count} ????? · ????????????",
43         "shotsTitle": "??? ?????? ?????????????? ?? (?????????) ?? ??? ?? ???? ??{conf}? ????
????? ???? ?? ?????? ?? ??????? ??????",
44         "shotsConfidence": " ? ??????: {conf}"
45     },
46     "footer": {
47         "count": "{count, plural, one {# ????} other {# ?????}}",
48         "summaryRest": "????? · ??? {total}",
49         "reelName": "??? ?? ???",
50         "reelNamePlaceholder": "my-highlights",
51         "build": "??? ????? ({count})",
52         "building": "? ? ? ? ?",
53         "floorOne": "<b>???? ?????? ?????? ?????</b> ??? {min} ?? ? ? ?????? ??? ? ???? ????
????? ???? , ?????? ?????? ???? ???? ?? ???? ???? ??????",
54         "floorAll": "<b>???? {count} ?????? ?????????</b> ??? {min} ?? ? ? ?????? ??? ? ???? ?? ?
?? ???? ???? ?? , ?????? ?????? ???? ???? ?????? ??????",
55         "floorSomeLead": "{count, plural, one {# ????} other {# ?????}}",
56         "floorSomeRest": "????? {min} ?? ? ? ???? ???? , ?????? ???? ???? ???? ?? ?????? (????
?? <code>min_shot_count</code> ????)"
57     },
58     "result": {
59         "ready": "???? ???? ???? ??",
60         "failed": "????? ??????",
61         "dismiss": "????? ??????",
62         "download": "{filename} ??????? ??????"
63     },
64     "build": {
65         "noVideo": "??? ?????? ?? ??? ???? ???? ?? ??? ?video=<video_id> ?? ?????? ? ???? ????
?????? ???? ?? ??????"
66     },
67     "selection": {
68         "count": "{count, plural, one {# ???? ?????} other {# ??????? ?????}}"

```

```
69      }
70    }
71
```

28. assistant (2026-06-21T09:17:48.204Z)

I now have the complete picture. Let me verify the kn catalog structure matches (for completeness of the i18n key conventions) and check the job `progress` shape and `result` shape to map progress UI accurately.

29. user (2026-06-21T09:17:52.034Z)

```

196:         progress TEXT,
486:     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
504:         "progress='starting' WHERE id = ?", (job_id,),
508:     def set_job_progress(self, job_id: str, progress: str) -> bool:
510:         "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
511:         f"Failed to set progress for job {job_id}",
524:         "UPDATE jobs SET status='done', progress='finished', result=?, "
559:     def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] = None)
-> bool:
566:         "progress=COALESCE(?, progress) WHERE id = ?",
567:         (f"+{max(0, int(delay_sec))} seconds", progress, job_id),
===progress payload shape in pipeline===
===result status field meaning===
135:         "status": "ok",
182:         "status": "failed",
199:         "status": "failed",
228:         "status": "failed",
241:         "status": "success",
299:     content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
313:     "status": job["status"],
426:     return {"status": "success", "filepath": out_path, "download_url": download_url}

```

30. user (2026-06-21T09:17:52.536Z)

```

1  {
2    "app": {
3      "badge": "?????-????? ???? . ????",
4      "title": "????? ?????????? ?????????????? ??????",
5      "intro": "<b>?????? ????</b>? ?????????? ??????? ???? ? ??? . <i>????? ?????? ????
15?</i> ? ??? ? ?????????????? ?????? ???? ?????????????? ????????. ?????? <b>????????????</b>
????????????????????; ?????????????????? ?????? ??????????????."
6    },
7    "lang": {
8      "label": "?????",
9      "en": "English",
10     "kn": "?????",
11     "hi": "??????"
12   },
13   "bulk": {
14     "selectAll": "????????? ??????????",
15     "clear": "?????????????",
16     "invert": "?????????"
17   },
18   "mode": {
19     "demo": "???? ???? ? ?????? ?????? ???? ?????? ?????????????????? ??????????????????
<code>?video=?video_id?</code> ??? ? ??????????",
20     "loading": "<code>{videoId}</code> ??? ?????????????? ??? ??????????????????",
21     "error": "<code>{videoId}</code> ??? ?????????????? ??? ??????????????: {error}",
22     "loaded": "? ?????????? <b>{count}</b> ?????????????? ??? ??????????."
23   },

```

31. assistant (2026-06-21T09:17:57.479Z)

I have everything I need. The job `progress` is a free-text TEXT column (e.g. "starting", "finished"), `status` is the execution state (queued|running|done|failed), and on done the `result` carries `video\_id` plus an inner `result.status` (success|failed). The download URL contract and all SPA patterns are confirmed.

Let me verify how dev proxy + offline detection works (vite.config + any offline banner) so I can address the offline-interplay risk accurately.

32. user (2026-06-21T09:17:58.449Z)

```
1      import { defineConfig } from "vite/config";
2      import react from "@vitejs/plugin-react";
3      import tailwindcss from "@tailwindcss/vite";
4
5      // The SPA makes SAME-ORIGIN relative `/api/*` calls. In the appliance (prod) a reverse
6      // proxy serves the SPA and proxies `/api` to the engine on 127.0.0.1:8000
7      // (docs/LOCAL_APPLIANCE_ARCHITECTURE.md). In dev, Vite proxies `/api` to the local
engine
8      // (`python -m backend.main --server`, :8000). Override with VITE_ENGINE_ORIGIN if
needed.
9      export default defineConfig({
10        plugins: [react(), tailwindcss()],
11        server: {
12          proxy: {
13            "/api": { target: "http://localhost:8000", changeOrigin: true },
14          },
15        },
16        test: {
17          // Default env is node (the pure lib/parser tests); the i18n render test opts into
jsdom via a
18          // `@vitest-environment jsdom` docblock. Match .test.tsx too so render tests are
discovered.
19          environment: "node",
20          include: ["src/**/*.test.{ts,tsx}"],
21          // DOM-env localStorage polyfill (jsdom under Node 26 ships none); node-env files
no-op.
22          setupFiles: ["src/test/setup.ts"],
23        },
24      });
25
```

33. user (2026-06-21T09:18:00.638Z)

```
./api/client.test.ts:92:    vi.stubGlobal("fetch", vi.fn(async () => { throw new
Error("offline"); }) as unknown as FetchFn);
./i18n/hygiene.test.ts:84:describe("offline / self-contained ? no external runtime assets", ()
=> {
./main.tsx:5:// Self-hosted OFL Noto fonts so Kannada/Hindi render crisply offline (tracker B4 /
decision D7) ?
===any router/navigate?===
./App.tsx:48:  const videoId = useMemo(() => new
URLSearchParams(window.location.search).get("video"), []);
```

34. assistant (2026-06-21T09:18:04.360Z)

I have a complete, code-grounded picture. There is no router (just `window.location.search` read once via `useMemo`), no existing offline banner (the SPA is "offline-resilient" only in the sense of self-contained assets + loud network errors via `ApiError`). The `videoId` is read once at mount and never set in-app. Now I have everything needed to produce the structured sketch.

35. user (2026-06-21T09:19:04.297Z)

Structured output provided successfully

36. assistant (2026-06-21T09:19:20.623Z)

I've mapped the B-P2 ingest+progress screen against the actual khelsutra SPA and the engine's confirmed contract, and returned the structured sketch. Key grounded findings:

- The SPA has **no router**: `App.tsx:48` reads `?video=` once via `useMemo` and never sets it. The single structural change is making `videoId` into state seeded from the URL, so the existing `useEffect([videoId])` load path (POST `/api/query ? render ? compile`) is reused as-is via `setVideoId` + `history.replaceState`.
- The engine contract is verified in `backend/main.py:260-322`: `POST /api/process` returns `202 {job_id, status:'queued', poll}`; `GET /api/jobs/{job_id}` returns `{status (queued|running|done|failed), progress (free-text), video_id, error, result}`. The two-layer status trap (job `status` vs inner `result.status`) is the main correctness risk.
- Two new client helpers (`processVideo`, `getJob`) drop cleanly into the existing `request<T>/ApiError/cid` pattern in `web/src/api/client.ts`; a new `useIngestJob` hook + `IngestPanel` component mirror the placement of `useRallySelection/QueryBar`.
- 16 new `ingest.*` i18n keys must land in all three of `en/kn/hi` `common.json` simultaneously, because `web/src/i18n/hygiene.test.ts` enforces cross-locales key parity.

The structured JSON (`currentFlow`, `apiClient`, `filesToTouch`, `i18nKeysNeeded`, `sketch`, `risks`) has been submitted.